

第六章 软件设计（概念及原则）

① 本章速览

核心问题

- 什么是软件设计？概要设计与详细设计各自做什么？
- 软件设计模型由哪两部分构成？静态 vs 动态结构的关系？
- 模块的 3 个基本属性、模块化的 5 条标准是什么？
- 信息隐藏（Parnas 1972）的核心思想？为什么能降低维护成本？
- 7 种内聚 由低→高的顺序？最高/最低各是什么？
- 7 种耦合 由低→高的顺序？最强/最弱各是什么？
- 为什么"作用范围必须在控制范围之内"？违反时如何优化？
- 七大 OO 设计原则 的核心思想（SRP/OCP/LSP/DIP/ISP/CARP/LoD）？
- 常见软件体系结构风格识别（管道/过滤器、调用/返回、分层、事件、C/S、解释器、仓库/黑板、MVC）？
- MVC 三要素的职责划分？Model / View / Controller 各自做什么？

核心结论

- 设计 = 把"做什么"（需求）转为"怎么做"（物理模型），输出设计模型 = 静态结构 + 动态结构。
- 概要设计 8 项活动：制订规范、系统结构设计、处理方式、数据设计、可靠性、界面、文档、评审。
- 详细设计 = 算法 + 内部数据结构 + 描述工具（流程图/PDL）。
- "高内聚低耦合"是模块独立性的核心；内聚越高→耦合越低→独立性越强。
- 内聚排序：巧合 < 逻辑 < 时间 < 过程 < 通信 < 顺序 < 功能（功能最强）。
- 耦合排序：非直接 < 数据 < 标记 < 控制 < 外部 < 公共 < 内容（内容最差）。
- OCP = "对扩展开放，对修改封闭"；LSP = "子类能替换父类"；DIP = "依赖抽象不依赖细节"。
- CARP = "先组合/聚合，后继承"；LoD = "只与直接朋友通信"。
- 设计模型 = 动态结构（交互图）→ 静态结构（设计类图）：先动态后静态。
- OOD 三项核心活动：体系结构设计 + 用例实现方案设计 + 用户界面设计。

高频考点（题库统计 2007–2024）

判断 单选 填空 简答 应用大题

- 7 种内聚 / 7 种耦合类型排序与判定（必考单选/判断，2007–2008、2010–2011、2018–2019）
- 七大 OO 设计原则（尤其 OCP、SRP、DIP、LoD、CARP）（必考判断/单选/简答，2008–2009、2009–2010、2011–2012、2023–2024）
- 降低耦合的方法 + 高内聚低耦合类型举例（必考简答，2011–2012、2012–2013）
- 概要设计 vs 详细设计任务区分 + 概要设计 8 项活动（必考简答/填空/判断，2010–2011、2021–2022、2023–2024）
- 模块作用范围 vs 控制范围（必考单选/简答，2010–2011、2011–2012、2012–2013）
- DFD→功能结构图（变换型/事务型）映射（必考应用大题，2009–2019 全覆盖）
- OOA vs OOD 区别 + OOD 工作与产出（必考简答/判断，2007–2008、2010–2011、2011–2012、2012–2013）
- MVC 三要素、五层体系结构（必考单选/简答，2013–2014、2018–2019）
- 体系结构风格识别（必考单选/判断，2018–2019）

② 题目关键词导航

题干关键词	考点定位	教材/PPT 锚点	典型题型
"为了提高模块独立性最好..."	数据耦合 / 功能内聚	§6.6.3 内聚与耦合	单选
"哪一种内聚独立性最弱"	巧合内聚最低	§6.6.3(1) 巧合内聚	单选
"模块作用范围应在控制范围之内"	不当调整 = 判定下移	§6.3.8 概要设计评审	单选
"开闭原则/可扩展不可修改"	OCP 对扩展开放对修改封闭	§6.7.2 OCP	判断/单选
"SRP 只有一个引起变化的原因"	SRP 定义	§6.7.1 SRP	判断
"依赖倒置含义"	高层不依赖低层，均依赖抽象	§6.7.4 DIP	简答
"违反迪米特法则"	"与陌生人说话" 火车式调用	§6.7.7 LoD	单选
"先组合/聚合后继承"	CARP 顺序	§6.7.6 CARP	判断
"内聚描述的是...的紧密程度"	内聚=模块内部；耦合=模块之间	§6.6.3	判断
"数据流图是哪个阶段工具"	DFD = 需求分析（非设计）	§6.5 设计模型	判断
"结构化软件设计是面向..."	面向数据流	§6.10 设计方法分类	单选
"OOA 与 OOD 表示法是否相同"	采用一致的 UML 表示法	§6.7 / §7.2.2	判断
"面向对象设计最关键的活动"	给对象分配职责	§7.3 GRASP	判断
"OO 概要设计不包括哪个图"	活动图（属详细设计）	§7.3 OOD 概要设计	单选
"OO 详细设计特指"	每个对象的方法定义	§7.3 OOD 详细设计	单选
"变换型步骤 / 事务型步骤"	传入-变换-传出 / 事务中心调度	§6.10 / §8 SA-SD	单选
"MVC 中模型的功能"	封装内核数据与状态	§6.9.5–7 MVC	单选
"五层体系结构"	界面/控制器/业务/持久化/系统	§7.2 模型层次化	简答
"降低耦合的方法"	选适当耦合类型/降接口复杂度/缓冲区	§6.6.5	简答
"DFD 转换为功能结构图"	变换型/事务型映射步骤	结构化设计 SD	应用大题

③ 知识主线

软件设计主线：需求分析（What）→ **概要设计**（系统结构 + 数据 + 处理方式）→ **详细设计**（算法 + 局部数据）→ 编码。

设计模型：静态结构（功能 + 数据结构）+ 动态结构（处理过程 + 协调机制）。

模块化主线：抽象 → 模块化 → 信息隐藏 → **模块独立性（内聚 + 耦合）** → 设计原则。

设计方法主线：① 面向数据流（结构化设计 SD：DFD → SC 图）② 面向对象设计 OOD（领域模型 + 用例模型 → 设计模型）。

OOD 设计模型主线：用例模型 + 领域模型（OOA 输入）→ **动态结构**（用例实现的顺序图/协作图）→ **静态结构**（设计类图，分层）→ 详细设计（方法 + 活动图/流程图）。

体系结构主线：体系结构 = {构件 + 连接件 + 约束} → 体系结构风格（管道/过滤器、调用/返回、对象、分层、事件、C/S、解释器、仓库/黑板、MVC）→ 异质体系结构。

④ 核心知识点详解（PPT 主导）

6.1 软件设计的目标

- 定义：根据软件需求分析的结果，设想并设计软件，即**根据目标系统的逻辑模型确定目标系统的物理模型**。
- 设计内容：软件系统的结构设计、处理方式（性能）的设计、数据结构和数据存储的设计、界面和可靠性设计等。
- 地位：**软件开发的基础和依据，是软件工程过程中的技术核心**，是后续开发步骤及软件维护工作的基础。
- 最终目标：取得**最佳方案**（Best Solution，主观）—— 在所有候选方案中，就节省开发费用、降低资源消耗、缩短开发时间的条件，选择具有**较高生产率、较高可靠性和可维护性**的方案。

★**评分点** 好的设计 ≠ 仅实现需求功能，还须满足性能、可靠性、可维护性、可扩展性等非功能需求。

6.2 软件设计的过程

软件设计 = 把软件需求变换成包含 **功能模型 + 数据模型 + 行为模型** 的过程。从工程管理角度分两阶段：

阶段	任务	产出
概要设计 (总体设计)	描绘可直接反映功能、数据、行为需求的软件总体框架	概要设计说明书、数据库设计说明书
详细设计 (过程设计)	对软件结构细化，得到各功能模块的 详细数据结构和算法 ，使模块在细节上接近源程序	详细设计说明书

从技术开发角度分三部分：

- 系统结构设计**：定义软件各主要元素（模块）之间的关系，包括模块接口设计。
- 数据设计**：各模块处理的数据 + 系统长久保存数据的数据结构 / 数据存储。
- 过程设计**：确定各功能模块内部结构的详细定义（算法逻辑 + 局部数据结构）。

图示信息（图 6-1 软件设计过程的两种表示）：
管理角度：需求分析 → [概要设计 ↔ 详细设计] → 编码；
技术角度：系统结构设计 → 数据设计 → 过程设计。两者通过“设计模型”贯通。

6.3 软件概要设计（8 项活动，必背）

- 制订设计规范**：阅读理解 SRS，确认需求可实现；选定设计方法（结构化/OO）；规定文档与编码标准。
- 软件系统结构设计**：① 划分模块层次结构 ② 确定每个模块功能 ③ 确定模块调用关系和接口 ④ 优化结构。
- 处理方式设计**（性能设计）：为每个模块定义算法并评估性能；性能指标 = **周转时间、响应时间、吞吐量、精度**。
- 数据设计**：① 输入输出文件的数据结构 ② 算法的逻辑数据结构 ③ 数据保护性设计（防卫性、一致性）。
- 可靠性设计**（质量设计）：长时间运行错误数 + 系统恢复能力。
- 界面设计**：直观反映系统功能，决定易用性。
- 编写概要设计文档**：概要设计说明书、数据库设计说明书、用户手册（修订）、测试计划。
- 概要设计评审**：可追溯性、接口（高内聚低耦合、作用范围 ≤ 控制范围）、风险、实用性、技术清晰度、可维护性、质量、备选方案、限制条件等 10 项。

6.4 软件详细设计

- 确定软件各功能模块内的**算法**及各功能模块的**内部数据结构**；
- 选定某种表达形式来描述各种算法（程序流程图 / PDL / 判定表）；
- 编写**软件详细设计说明书**；
- 进行详细设计的评审。

6.5 软件设计模型

软件设计**既是过程又是模型**。设计模型由两部分构成：

构成	含义	对应活动
动态结构设计	表示功能响应客户请求时处理数据的过程或条件，解释各功能如何协调工作	过程设计、接口设计
静态结构设计	由软件的功能结构和数据结构组成，展示满足所有需求的框架结构	系统结构设计、数据结构设计

设计活动共 6 类：① **系统结构设计**（体系结构 + 功能结构） ② **数据设计**（存储 + 局部） ③ **接口设计**（模块 + 子系统） ④ **过程设计**（内部逻辑 + 处理能力） ⑤ **组件设计** ⑥ **结构优化设计**。

6.6 软件设计的一般性原则（Davis 等）

- 设计不应有 tunnel vision（管状视野）；要 traceable to the analysis model（可追溯到分析模型）。
- 不要重新发明轮子**（not reinvent the wheel）。
- 最小化软件与现实问题的“智力距离”；体现统一性与集成性。
- 结构要适应变化**（accommodate change）；优雅降级（degrade gently）—— 异常保护机制。
- Design is not coding, coding is not design**（设计不是编码）。
- 设计过程中即评估质量，而非事后；评审以最小化概念（语义）错误。

衡量设计过程的技术原则：可追踪可回溯；实现分析模型所有显式需求；满足隐式需求（安全、数据完整）；文档可读可理解，易编程、易测试、易维护。

衡量设计模型的技术原则：展现软件全貌；是**分层结构**（用可识别设计模式搭建）；由具备良好设计特征的构件构成；可演化实现；**模块化**（独立功能构件）；接口降低与外部复杂连接；导出合适的数据结构。

6.6.1 软件模块化

- 模块定义：整个软件可被划分成若干可单独命名且可编址的部分。
- 模块 3 个基本属性：功能（做什么）、逻辑（怎么做）、状态（使用环境和条件）。
- 模块外部特性：模块名、参数表、给系统造成的影响；内部特性：完成功能的代码 + 内部数据。
- 模块化的优点：降低系统复杂度，使系统易修改；推动并行开发，提高生产效率。
- 模块设计的 5 条标准：① 可分解性 ② 可组装性 ③ 可理解性 ④ 连续性（小变更只动一个模块） ⑤ 保护性（异常局限在模块内）。

图 6-2 模块大小、模块数目与费用的关系（曲线图）：
横轴 = 模块数 M；纵轴 = 软件总成本。
曲线先降后升：模块小时单个模块工作量低，但接口工作量大；存在一个最佳模块数 M 使总成本最小。
结论：模块数既非越多越好，也非越少越好，应处于 M 附近。

6.6.2 信息隐藏（Parnas, 1972）

- 定义：每个模块的实现细节对于其它模块来说是隐蔽的；模块所包含的信息（包括数据和过程）不允许其它不需要这些信息的模块使用。
- 最大好处：使得模块的修改和软件的维护所造成的影响局限在一个或几个模块范围内。
- 判定标准：模块之间的关系能够达到信息隐藏，就可以认为是合理的模块划分。

6.6.3 模块的独立性（Myers, 1978）

功能独立性是抽象、模块化和信息隐藏的直接产物。1978 年 Myers 在《Composite/Structured Design》提出两条准则：**内聚 + 耦合**。

模块的内聚（Cohesion）：模块功能强度（一个模块内部各元素彼此结合的紧密程度）的度量。**内聚越高** → 与其他模块耦合越低 → **独立性越强**。

模块的耦合（Coupling）：模块之间**相对独立性**（互相连接的紧密程度）的度量。耦合越高 → 独立性越弱。

"高内聚低耦合"是模块独立性强的标志。

内聚 7 种类型（由低→高排序，必背）

类型	特征	判定要点
① 巧合内聚	几段相同代码为节省空间凑成模块，无独立功能	内聚最低，难理解、难维护
② 逻辑内聚	几种相关功能组合，由控制参数决定执行哪个	传控制标志→增加耦合
③ 时间内聚	各功能必须在同一时间段执行（如初始化、终止模块）	经典内聚
④ 过程内聚	按流程图划分的次序执行（循环/判定/计算分块）	仅含完整功能的一部分
⑤ 通信内聚	各功能使用 相同输入数据 或产生 相同输出数据	可通过 DFD 判定；又称信息内聚
⑥ 顺序内聚（序列内聚）	前一子模块输出 = 后一子模块输入	数据在模块内顺序流转
⑦ 功能内聚	所有部分为同一具体功能协同工作，不可分割	内聚最高、最佳

★评分点 考过：内聚最低=巧合；最高=功能；"通信内聚"="信息内聚"；功能内聚是 Copy 这种"只做一件事"的模块。

耦合 7 种类型（由低→高排序，必背）

类型	特征	判定要点
① 非直接耦合	两模块无直接关系，仅通过上级模块的控制和调用	独立性最强（最低耦合）
② 数据耦合	通过 简单数据参数 （非控制、非公共数据）交换信息	松散、安全、可靠
③ 标记耦合	通过参数表传递 记录/数据结构子结构 ，非简单变量	又称数据结构耦合/特征耦合
④ 控制耦合	通过 开关、标志、名字 等控制信息明显控制选择另一模块功能	单一接口选多功能
⑤ 外部耦合	都访问同一 全局简单变量 （非通过参数表）	共享通信协议或设备接口

⑥ 公共耦合	都访问同一公共数据环境（全局数据结构、共享通信区）	多模块共占、影响大
⑦ 内容耦合	直接访问另一模块 内部数据 、不通过正常入口、代码重叠、多入口	病态耦合，独立性最弱

★评分点 考过：耦合最低=非直接；"模块间最好 = 数据耦合"；最强/最差=内容耦合。

6.6.4 模块耦合度计算（公式）

$$\text{Coupling}(C) = 1 - \frac{1}{d_i + 2c_i + d_o + 2c_o + g_d + 2g_c + w + r}$$

- 数据/控制参数： d_i 数据输入个数； c_i 控制输入个数； d_o 数据输出个数； c_o 控制输出个数。
- 全局耦合项： g_d 数据全局变量数； g_c 控制全局变量数。
- 环境耦合项： w 扇出数（调用模块数）； r 扇入数（被调用模块数）。
- 取值范围：约 0.67（低耦合）~ 1.0（高耦合）。值越大耦合越强。

6.6.5 降低模块耦合度的方法（必背简答）

- 根据问题特点选择适当的耦合类型**：传送数据信息优于控制信息；模块调用方式优先用 call（不直接引用）；错误处理可集中或分散。
- 降低模块接口的复杂度**：减少参数数量；用 call 代替"直接引用"；尽量少用指针/过程类型参数；以标准方式而非嵌套方式传信息。
- 将模块的通信信息放在缓冲区中**：缓冲区是 FIFO 队列，减少模块间互相等待。

★评分点 官方列举的 6 种降耦合方法（每答对 1 个 1 分，3 分即可）：① 多传数据，少传控制 ② 降低接口复杂度 ③ 减少接口参数数量 ④ 用 call 代替直接引用 ⑤ 简单数据结构代替标记耦合 ⑥ 通信信息放缓冲区。

6.7 面向对象设计七大原则（必考核心）

原则	缩写	一句话核心	关键词
单一职责	SRP	就一个类而言，应该仅有一个引起它变化的原因	职责=变化原因
开闭原则	OCP	可以扩展，但不能修改；对扩展开放、对修改封闭	抽象+多态
里氏替换	LSP	子类应当可以替换父类并出现在父类能够出现的任何地方	行为一致
依赖倒置	DIP	高层不应依赖低层，二者都应依赖"抽象"	依赖抽象
接口隔离	ISP	采用多个特定客户类接口，比通用接口好	按客户拆接口
组合/聚合复用	CARP	新对象使用已有对象、向其委托责任；优先组合，后继承	黑盒复用
迪米特法则	LoD	最少知识原则：一个对象应尽可能少了解其它对象	只与朋友说话

6.7.1 单一职责原则（SRP）

- 定义：就一个类而言，应该仅有一个引起它变化的原因。"职责" = "变化的原因"。
- 核心：一个职责 = 一个变化原因。一个类承担多个职责意味着多个变化原因相互耦合，应分离到不同类中。
- 示例：Rectangle 类有 getArea()（业务）和 draw()（GUI）两职责 → 应拆分为 GeometricRectangle + RectangleFigure，避免业务应用被迫依赖 GUI。

6.7.2 开闭原则（OCP）

- 定义：软件实体（类、模块、函数）可以扩展，但不可修改。
- 两个特征：① 对扩展开放（Open for extension） ② 对修改封闭（Closed for modification）。
- 实现关键：使用抽象，识别共性和变化点，用封装处理变化。
例：Phone/Email → 抽象为 ContactMech 超类，新增 Fax 只需新增子类，不修改 ContactManager。

★评分点 考过：OCP = "对扩展开放，对修改封闭"；只能添加新功能，不能改原有代码。

6.7.3 里氏替换原则（LSP）

- 定义：子类应当可以替换父类并出现在父类能够出现的任何地方（Liskov, 1987）。
- 形式化：若 S 是 T 的子类型，则所有 T 编写的程序 P 中，用 S 的对象替换 T 的对象后，P 行为不变。
- 核心：所有子类行为功能必须与客户类对其父类所期望的行为功能保持一致。
- OCP vs LSP：OCP 强调对变化的类进行抽象；LSP 强调实现抽象化的具体规范。

6.7.4 依赖倒置原则（DIP）

- 核心定义：① 高层模块不应依赖低层模块，二者都应依赖"抽象"。② "抽象"不应依赖细节，细节应依赖"抽象"。
- "倒置"双重含义：依赖关系倒置 + 接口所有权倒置（通常是客户类拥有抽象接口）。
- 启发式规则：程序中所有依赖关系应终止于抽象类或接口——任何变量不应持有指向具体类的指针/引用；任何类不应从具体类派生；任何方法不应覆盖基类已实现的方法。
- 意义：DIP 是框架设计的核心原则，能提高高层模块的可复用性、降低层次耦合。

6.7.5 接口隔离原则（ISP）

- 定义：采用多个与特定客户类有关的接口，比采用一个通用的涵盖多个业务方法的接口要好。
- 核心：服务器类为多个客户类提供不同服务时，应为每个客户类创建特定业务接口，而非统一接口。
- 通俗：向一个客户提供超过客户要求的服务承诺时，给服务提供方带来不必要的维护负担。

6.7.6 组合/聚合复用原则（CARP）

- 定义：在一个新对象里使用一些已有对象，使之成为新对象的一部分；新对象通过向已有对象委托部分责任达到复用。
- 核心：实现复用时应首先使用"组合/聚合"，其次才考虑"继承"。严格遵循 LSP 时才用继承。
- 判断：has-a 用组合/聚合，is-a 用继承。

对比	组合/聚合复用	继承复用
----	---------	------

封装性	支持（黑盒复用）	破坏（白盒复用）
灵活性	运行期动态引用	静态、缺乏灵活
耦合度	通过接口低耦合	父类改子类必改
上下文依赖	不依赖上下文	同上下文环境
LSP 保障	不破坏 LSP	可能破坏 LSP

6.7.7 迪米特法则（LoD / 最少知识原则）

- 定义：一个对象应当尽可能少地了解其它对象；只与直接的朋友通信，不与"陌生人"说话。
- "朋友"定义：① 当前对象本身（this） ② 以参量形式传入当前对象方法的对象 ③ 当前对象的实例变量直接引用的对象 ④ 实例变量若为聚集，则聚集集中的元素 ⑤ 当前对象所创建的对象。
- 典型违反：火车式调用 a.b().c().d() —— 层层调用访问无直接关联的底层类。

★评分点 迪米特法则（LoD）直接约束模块/对象间的耦合度，是描述"模块耦合性"的 OO 设计原则（2023-2024 真题考过）。

6.8 软件设计基础

6.8.1 自顶向下，逐步细化（Wirth 提出）

将软件体系结构按自顶向下方式，对各个层次的过程细节和数据细节逐层细化，直到能用程序设计语言实现。

6.8.2 系统控制结构

- 控制层次用程序的层次（树形或网状）结构表示；顶层模块 = 主模块。
- 深度 = 程序结构的层次数（反映规模/复杂度）；宽度 = 同层模块的最大个数。
- 扇出 = 一个模块直接调用的其它模块数目（多扇出 = 控制协调多下属模块）；扇入 = 调用某模块的模块数（多扇入 = 公用模块）。

6.8.3 结构划分 + 结构图

- 水平划分：按主要程序功能定义模块结构的各分支（输入/处理/输出）。优点：易修改、易扩充；缺点：通过模块接口传递更多数据。
- 垂直划分（因子划分）：自顶向下分布——顶层控制，低层执行输入/计算/输出。优点：低层修改不传播副作用。
- 结构图（SC 图）符号：矩形框=模块；单向箭头=调用（调用方→被调用）；带空心圆尾短箭头=数据信息；带实心圆尾短箭头=控制信息；菱形=条件调用；弧形=循环调用。

★评分点 考过：“软件结构图中直线连接表示什么关系”→ 调用关系。

6.8.4 数据结构

标量项 → 顺序向量（一维数组）→ n 维向量空间（二维矩阵）→ 链表 → 多重链表（分层/网络结构）。栈、队列等逻辑结构可用向量或链表实现。

6.8.5 软件过程

程序结构描述整体控制层次；软件过程则描述每个模块的处理细节：事件顺序、判定点、重复操作、数据组织。软件过程也是层次化的。

6.9 软件体系结构

6.9.1 概述

大规模复杂系统，总体结构设计比数据结构/算法更重要。研究内容：体系结构描述、风格、评价、形式化方法。根本目的：解决软件重用、软件质量、软件维护问题。

6.9.2 体系结构定义（4 大定义）

定义	核心公式
Booch/Rumbaugh/Jacobson	体系结构 = {组织, 元素, 子系统, 风格}
Bass	系统结构 = {软件构件, 外部可视属性, 构件关系}
Shaw（4 模型）	结构模型 / 框架模型 / 动态模型 / 过程模型
Garlan & Shaw（主流）	体系结构 = {构件, 连接件, 约束}

6.9.3 三要素（必背）

软件体系结构三要素：① 程序构件（模块）的层次结构 ② 构件之间交互的方式 ③ 数据的结构。

体系结构设计应保持 3 个性质：结构、附属功能、可复用。

构件分类（表 6-1）

构件	特点和示例
纯计算构件	简单输入/输出，无状态变化。如数值计算、过滤器、转换器
存储构件	存放共享、永久、结构化数据。如数据库、文件、符号表
管理构件	操作与运行状态紧耦合。如 ADT、对象、服务器
控制构件	管理其它构件运行时间/时机/次序。如调度器、同步器
链接构件	实体间传递信息。如通信机制、用户界面

连接方式（表 6-2）

连接	特点与示例
过程调用	普通过程调用（同命名空间）/ 远程过程调用（异命名空间）
数据流	UNIX 管道 pipes
间接激活	事件驱动系统、自动垃圾回收
消息传递	TCP/IP，同步或异步
共享数据	多用户数据库、数据黑板系统

6.9.5 体系结构风格（核心）

体系结构风格 = 描述某一特定应用领域中系统组织方式的惯用模式。四要素：① 词汇表 ② 配置规则 ③ 语义解释原则 ④ 对基于此风格的系统所进行的分析。

① 管道/过滤器风格（Pipe and Filter）

- 构件 = 过滤器（filter）：对输入流处理、转换，递进处理（可在全部输入接受完前开始输出），可并行。
- 连接件 = 管道（pipe）：信息流的导管。
- 典型例子：UNIX shell（cat file | grep xyz | sort | uniq > out）、编译器（词法→句法→语义→代码生成）。
- 优点：隐蔽性、高内聚低耦合、支持重用、维护简单、支持并行。
- 缺点：导致批处理结构；不适合交互应用；数据同步困难；解析/合成数据开销。

② 调用/返回风格

大型软件开发主流风格，包含 3 种子风格：

- 主/子程序风格：结构化程序设计；单线程控制；分层推理。
- 对象风格：基于数据抽象和面向对象；封装、黑箱操作；对象耦合较紧（标识改变牵连调用方）。
- 分层风格：每层为上层提供服务、作为下层客户；只影响相邻两层；支持抽象递增、功能增强、复用。

③ 基于事件风格（隐式调用）

构件不直接调用过程，而是声明/广播事件；其他构件把过程注册为与事件相关联。适用于低耦合构件集合、动态重配置应用。优点：广播者无需知道被影响构件；缺点：放弃主动控制、推理困难。

④ 客户端/服务器风格（C/S）

目标 = 可测量性；适用于数据/处理分布、构件通过网络连接。变体：代理风格（CORBA/SOAP/Web Service）和 P2P 风格（X Window）。

⑤ 解释器风格（虚拟机）

用于建立虚拟机，弥合程序语义与硬件差异。例：Java/Smalltalk 编译器、Prolog 专家系统、Awk/Perl 脚本语言。由 4 构件构成：被解释的程序、执行引擎、程序状态、引擎状态。

⑥ 仓库风格 + 黑板风格

- 仓库风格：中央数据结构（当前状态）+ 独立构件集合（操作中央数据）。
- 两种控制方法：① 输入事务选择处理 = 传统数据库体系结构；② 中央数据状态决定处理 = 黑板体系结构。
- 黑板风格三部分：知识源、黑板数据结构、控制器。适用于信号处理、语音识别、模式识别、三维分子建模。

⑦ MVC 风格（必考）

构件	职责
视图 View	为用户显示模型信息；从模型获取数据；一个模型可对应多个视图
模型 Model	应用程序核心；封装内核数据与状态；对模型的修改扩散到所有视图
控制器 Controller	提供用户操作接口；接受用户输入，将输入事件转换成服务请求传送到模型或视图

MVC 运行机制示意：用户 → Controller（输入事件）→ Model（业务处理+状态变更）→ 通知所有已注册的 View → View 从 Model 取数据并刷新显示。M 和 V 的代码分离，同一个 Model 可挂多个 View。

优点：问题分解、简化设计、可扩展；改界面不影响内核；易于演化、可维护性好；运行时可改变。缺点：主要局限于用户界面开发领域。

⑧ 异质体系结构

组合多种风格的 3 种方法：① 空间异质（不同子系统用不同风格）② 时间异质（不同时间采用不同风格）③ 层次异质（构件/连接件内部可以是另一种风格）。

6.10 软件设计方法分类

- ① 面向数据流的设计（过程驱动，以结构化设计 SD 为代表）：DFD → SC 图。
- ② 面向对象的设计 OOD：领域模型 + 用例模型 → 设计模型。

⑤ 补充知识点（教材独有）

补充 1: DFD → 功能结构图映射（结构化设计 SD 核心）

结构化设计（SD）以 DFD（数据流图）为输入，通过变换分析或事务分析将 DFD 映射为模块结构图（SC 图）。

变换型 DFD（Transform Center）映射步骤

1. 复查/精化 DFD；
2. 确定 DFD 类型（变换型还是事务型）；
3. 确定输入流和输出流边界，划分出传入路径 / 主干变换 / 传出路径三段（边界划分不唯一）；
4. 建立初始 SC 图：顶层 = 主模块；一级 = "取得输入"模块 + "主干变换"模块 + "给出输出"模块；二级 = 沿 DFD 中各加工递归展开；
5. 优化 SC 图（启发式规则：高扇入、低扇出、作用范围 ≤ 控制范围）。

标准三步工作流程：① 取得数据（传入）② 变换数据（主干）③ 给出数据（传出）。"计算数据"不是变换型独立步骤。

事务型 DFD（Transaction Center）映射步骤

1. 识别事务中心：一个加工接收一个输入流，经调度后发出多个并行输出流，该加工即为事务中心。
2. 确定逻辑输入流和逻辑输出流；
3. 映射为 SC 图：顶层 = 主模块；一级 = "取得输入"模块 + 调度模块（事务分发器）+ 若干"给出输出"模块；调度模块下挂各事务分支（带菱形选择符号）。

变换型 vs 事务型（必考对比）

对比维度	变换型（Transform）	事务型（Transaction）
数据流特征	输入流 → 主干加工 → 输出流（线性三段）	一个输入 → 事务中心 → 多个并行输出分支
中心加工	主干变换（变换中心）	事务中心（调度分发）
映射主模块	主模块 + 取得/变换/给出模块	主模块 + 取得模块 + 调度模块 + 各事务分支
分支选择	无（线性流程）	有（菱形选择符号）
典型例子	数据处理系统（输入-处理-输出）	菜单驱动、ATM、挂号选科

混合型 DFD（应用大题套路）

题干虚线框中间部分 = 中心；先判断中心类型（变换/事务），再按对应规则映射。事务中心必有调度模块 + 菱形选择；变换中心必有"取得""给出"对称结构。

补充 2: 面向对象设计（OOD）补充

OOD 三项核心活动

1. 软件体系结构设计：参考分层结构（如 MVC、五层体系结构），输出 UML 包图。
2. 用例实现方案设计：针对每个用例绘制交互图（顺序图/协作图），归纳出设计类图。
3. 用户界面设计。

OOD 五层软件分层体系结构（必背）

1. 用户界面层（GUI）
2. 控制器层（Controller / 处理层）
3. 业务逻辑层 / 领域层（Domain）
4. 持久化层（数据访问层）
5. 系统层（系统管理类）

分层好处：① 增加软件健壮性，易于扩展和维护 ② 增加软件可移植性 ③ 把消息流动限制在一个方向上，减少类与类之间的耦合。

OOD 设计模型的两个产出

- 动态结构：每个用例的用例实现（顺序图/协作图），描述软件对象协作。
- 静态结构：在所有动态结构基础上确定对象协作关系，用设计类图表示（含属性 + 方法 + 关系 + 分层）。

★评分点 OOD 设计模型 = 动态结构（顺序图/协作图）+ 静态结构（设计类图）+ 状态图（复杂对象）+ 活动图（详细设计方法内部逻辑）。"先动态、后静态"——动态结构是静态结构的前提。

OOA vs OOD 区别（必考简答）

维度	OOA（分析）	OOD（设计）
----	---------	---------

面向	问题域（现实世界）	具体实现（解域）
目标	把需求转为面向对象概念描述的分析模型	产生符合实现条件的设计模型
表示法	UML	UML（与 OOA 一致）
典型产出	用例模型 + 领域模型	设计模型（动态+静态）

OOD 相对结构化的优势：OOA 和 OOD 采用一致的表示法，从 OOA 到 OOD 不存在转换，紧密衔接，大大降低过渡难度、工作量、出错率。

OOD 已知条件（输入）+ 设计结果（输出）

输入：用例模型（用例图+用例描述+SSD+操作契约）、领域模型、软件体系结构框架、OO 设计原则/设计模式。

输出：动态结构（用例实现的顺序图/协作图）+ 静态结构（设计类图/包图/组件图）+ 状态迁移图 + 用户界面设计。

补充 3: 模块控制范围 vs 作用范围（必考）

- 控制范围：模块本身 + 其所有下属模块（包括直接和间接调用的下级模块）。
- 作用范围：受该模块内判定语句影响的所有模块（包括判定所在模块本身）。
- 核心原则：作用范围必须在控制范围之内（6.3.8 评审清单的硬性要求）。

违反时的优化策略：① 将判定语句上移到作用范围的最近公共祖先层；② 将受影响模块下移到控制范围内；③ 将判定所在模块合并到父模块使判定处于较高层次。反例："将判定下移到最低层次"是错误做法。

补充 4: GRASP 设计模式（简要）

- 控制器（Controller）模式：把系统事件分配给控制器对象（用例控制器 / 全局控制器）。
- 信息专家（Information Expert）模式：把职责分配给拥有完成该职责所需信息的对象。
- 创建者（Creator）模式：B 创建 A 当且仅当 B 包含/聚合 A、B 记录 A、B 密切使用 A、B 拥有 A 的初始化数据。

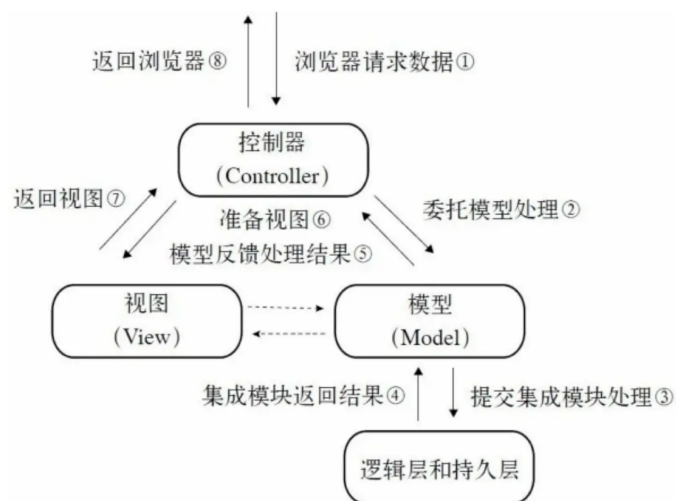
注：GRASP 属于 OOD 微观职责分配模式，与 7 大设计原则共同指导 OOD。详见 ch07。

补充 5: 操作契约后置条件（与 OOD 关系）

操作契约后置条件描述领域模型中概念类对象的状态变化（创建/消除、关联形成/断开、属性值修改）。在 OOD 阶段，概念类对象被转换为软件对象（设计类），并新增控制器、DAO 等设计类。

补充 6: UML 设计阶段常用图

类图、对象图、顺序图、协作图、状态图、活动图、包图、组件图、部署图。OO 概要设计核心：交互图（动态）+ 设计类图（静态）+ 状态图（复杂对象）；详细设计：活动图/流程图（方法内部逻辑）。



MVC 风格

⑥ 核心对比表

对比 1：概要设计 vs 详细设计

维度	概要设计	详细设计
别名	总体设计	过程设计
目标	总体框架（功能+数据+行为模型）	模块内部算法 + 局部数据结构
关键活动	8 项（见 6.3）	算法、数据结构、描述工具、评审
抽象层次	高层（模块划分、接口）	低层（接近源代码）
关注问题	What & Where	How
文档	概要设计说明书	详细设计说明书

对比 2：7 种内聚（低→高）+ 7 种耦合（低→高）

排序	内聚	耦合
最低	巧合内聚	非直接耦合
2	逻辑内聚	数据耦合
3	时间内聚	标记耦合
4	过程内聚	控制耦合
5	通信内聚	外部耦合
6	顺序内聚	公共耦合
最高	功能内聚（最佳）	内容耦合（最差）

口诀：内聚从巧合到功能（低→高）；耦合从非直接到内容（低→高）。内聚越高越好，耦合越低越好。

对比 3：OO 设计原则核心区别

原则	针对	核心	易混淆点
SRP	类	单一变化原因	≠ 只有一个方法
OCP	扩展/修改	扩展开、修改闭	≠ 不能修改 bug
LSP	继承	子类可替换父类	是 OCP 的补充
DIP	依赖关系	依赖抽象非细节	框架设计核心
ISP	接口	多个专用接口	≠ 不能用统一接口
CARP	复用	先组合后继承	has-a vs is-a
LoD	耦合	最少知识	= 迪米特法则

对比 4：变换型 vs 事务型

维度	变换型	事务型
DFD 特征	线性：传入→变换→传出	放射：1 输入 → 中心 → N 输出
中心类型	变换中心	事务中心（调度）
调度模块	无	有（菱形选择）
三步流程	取得/变换/给出数据	取得/调度分发/给出
边界选择	不唯一	事务中心明确

对比 5：常见软件体系结构风格

风格	构件/连接件	典型应用
管道/过滤器	过滤器 + 管道	UNIX shell、编译器
主/子程序	主程序 + 子程序	结构化程序
对象风格	对象 + 方法调用	面向对象系统
分层风格	层 + 协议	OSI、操作系统
基于事件	构件 + 事件广播	IDE、消息总线
C/S	客户端 + 服务器	Web、分布式
解释器	引擎+程序+状态	Java VM、Prolog
仓库/黑板	中央数据 + 操作构件	DB、语音识别
MVC	Model+View+Controller	用户界面

对比 6：模块控制范围 vs 作用范围

概念	定义	判定
控制范围	模块本身 + 所有下属模块	由调用层次决定
作用范围	受判定影响的模块集合	由数据流和控制流决定
规则	作用范围 ≤ 控制范围；判定模块应靠近受影响模块	

⑦ 开放题答题模板

Q1: 软件设计为何分概要设计和详细设计两阶段?

答题步骤:

1. **管理角度**: 分阶段评审和控制, 先做总体框架 (概要), 再细化各模块 (详细), 降低每阶段复杂度。
2. **技术角度**: 先系统结构设计 → 数据设计 → 过程设计 (图 6-1); 先解决"系统如何整体实现", 再解决"每个模块如何做"。
3. **抽象层次**: 概要设计抽象层次高 (便于宏观把握), 详细设计抽象层次低 (接近源代码), 便于编码实现。
4. **设计评审**: 概要设计完成后评审, 确保结构合理、可追溯, 再进入详细设计阶段。

Q2: DFD 与功能结构图的关系 + 如何将 DFD 转换为功能结构图?

关系: DFD 是需求分析阶段的工具模型 (描述"做什么"); 功能结构图是概要设计阶段的工具模型 (描述"如何做")。DFD 是 SC 图的输入。

转换步骤 (必背):

1. **复查并精化 DFD**, 确保 DFD 完整、正确。
2. **确定 DFD 类型**: 变换型 (线性传入-变换-传出) 还是事务型 (一个输入放射出多个分支)。
3. **划分边界**: 变换型 → 找出传入路径、主干变换、传出路径; 事务型 → 找出事务中心 + 调度逻辑。
4. **建立初始 SC 图**: ① 顶层主模块; ② 一级: 取得输入 + 中心 (变换/调度) + 给出输出; ③ 二级及以下: 按 DFD 各加工递归展开。
5. **优化 SC 图**: 高扇入、低扇出、作用范围 ≤ 控制范围、消除内容/公共耦合。

Q3: 模块的耦合性定义 + 至少 3 种降低耦合的方法

耦合性定义: 模块之间相对独立性 (互相连接的紧密程度) 的度量。

降低耦合的 6 种方法 (任选 3 种即可):

1. 模块间多传递**数据信息**, 尽量减少和避免传递**控制信息**;
2. 降低模块接口的复杂性 (参数数量、调用方式、信息结构);
3. 减少接口信息传递数量;
4. 以系统调用 (call) 方式代替直接引用;
5. 传递信息的结构尽量简单 (以数据耦合代替标记耦合);
6. 把模块的通信信息放在缓冲区中。

Q4: 什么是模块的信息隐藏? 两种高内聚 + 两种低耦合类型

信息隐藏: 每个模块的实现细节对其它模块是隐蔽的; 模块所包含的信息 (数据和过程) 不允许其它不需要这些信息的模块使用。

两种高内聚: 功能内聚、顺序内聚 (或信息/通信内聚)。

两种低耦合: 非直接耦合、数据耦合。

Q5: OOA 与 OOD 的不同 + 相对结构化的优势

不同:

- 分析阶段针对问题域 (现实世界), 把需求转为面向对象概念描述的分析模型。
- 设计阶段考虑具体实现, 产生符合实现条件的设计模型。

相对结构化的优势: OOD 模型以 OOA 模型为基础, 且 OOA 和 OOD 采用一致的表示法 (都是 UML), 从 OOA 到 OOD 不存在转换, 紧密衔接, 大大降低过渡难度、工作量、出错率。

Q6: 为什么限制模块的作用范围在控制范围内?

核心原则: 模块的作用范围必须在控制范围之内。

1. **保证可控**: 作用范围超出控制范围 → 上层判定无法直接影响下层模块, 需通过参数层层传递, 增加耦合和复杂度。
2. **降低耦合**: 违反时往往伴随控制耦合, 判定结果通过开关/标志在模块间传播。
3. **便于维护**: 修改判定逻辑时影响范围清晰, 不传播到无关模块。
4. **提高可读性**: 判定位置和受影响模块在同一控制子树内, 易于理解。

违反时的优化: ① 判定上移到公共祖先; ② 受影响模块下移到控制子树; ③ 合并判定所在模块到父模块。**错误做法**: 判定下移到最低层次。

Q7: OOD 需要做哪些工作? 对应成果模型?

三方面工作:

1. 软件体系结构设计 → 成果: UML 包图 (如五层分层结构);
2. 用例实现方案设计 → 成果: 交互图 (顺序图/协作图) + 设计类图 + 复杂对象的状态图;
3. 用户界面设计。

详细设计: 每个对象的方法使用活动图/程序流程图进行的内部逻辑设计。

Q8: UML 设计阶段至少 4 种图

类图、对象图、顺序图 (时序图)、协作图、状态图、活动图、包图、组件图、部署图 (任 4+)。

Q9: 五层软件分层体系结构 + 模型层次化好处

五层: 界面层、控制器/处理层、业务/领域层、持久化层、系统层。

好处: ① 增加软件健壮性, 易于扩展和维护; ② 增加软件可移植性; ③ 把消息流动限制在一个方向上, 减少类间耦合。

Q10: 依赖倒置原则 (DIP) 的含义

1. **核心定义**: ① 高层模块不应依赖低层模块, 二者都应依赖"抽象"。② "抽象"不应依赖细节, 细节应依赖"抽象"。
2. **"倒置"双重含义**: ① 依赖关系倒置 (低层依赖高层定义的抽象接口); ② 接口所有权倒置 (客户类拥有抽象接口)。
3. **启发式规则**: 所有依赖关系应终止于抽象类或接口——变量不持有具体类指针/引用; 类不从具体类派生; 方法不覆盖基类已实现方法。
4. **意义**: 是框架设计的核心原则, 提高高层模块的可复用性、降低层次耦合。

⑧ 历年真题汇总与详解

说明：以下答案严格依据教材 §6 概念重写，原题库自带答案若有偏差一律以教材原文为准。综合大题/画图应用题不在本节范围。

一、判断题

【2008–2009 期末】好的软件设计是指按照该设计方案能够实现需求定义的系统功能。

× 错误。

解析：好的设计不仅要实现功能需求，还要满足性能、可靠性、可维护性、可扩展性等**非功能需求**。仅以“实现系统功能”定义过于片面。

【2008–2009 期末】SRP 原则阐述的是对于一个类而言，应该只有一个引起它变化的原因。

√ 正确。

解析：§6.7.1 SRP 原文——“就一个类而言，应该仅有一个引起它变化的原因”，将“职责”定义为“变化的原因”。

【2008–2009 期末】面向对象设计中最关键的活动是找到对象并给对象分配职责。

√ 正确。

解析：OOD 最关键的活动是**正确地给对象分配职责**，直接关系到设计模型的质量。GRASP 模式（控制器、创建者、信息专家）用于系统化指导职责分配。

【2009–2010 期末】面向对象设计中的开闭原则，要求对已有的软件类可以进行修改，而不能添加新的需求。

× 错误。

解析：OCP 是“对**扩展开放**、对**修改封闭**”——通过扩展（新增类/方法）实现新功能，**不修改**已有代码。题干恰好相反。

【2010–2011 期中】**面向对象设计中最关键的活动是找到对象并给对象分配职责。**

√ 正确。

解析：同上。“找到对象并分配职责”正是 OOD 的核心。

【2010–2011 期中】OOA 和 OOD 分别采用不同的概念和表示法。

× 错误。

解析：**OOA 与 OOD 采用一致的面向对象概念和 UML 表示法**，二者无缝衔接。题干与教材相反。

【2010–2011 期末 B】内聚性高的模块其耦合性也高。

× 错误。

解析：§6.6.3 原文——“**高内聚低耦合**”。内聚程度越高，模块间的耦合程度就越低，故“高内聚导致高耦合”说法错误。

【2010–2011 期末 B】数据流图是结构化设计阶段的工具模型，表明系统“如何做”。

× 错误。

解析：**DFD 是结构化分析阶段（需求分析）的建模工具，描述系统“做什么”**（数据流动与加工）。结构化设计阶段的工具是 SC 图（功能结构图）。

【2010–2011 期末 B】模块划分得越小，总的软件开发成本就越小。

× 错误。

解析：§6.6.1 图 6–2 模块大小、模块数目与费用的关系曲线——存在一个最佳模块数 M 使总成本最小。**一味细分导致接口工作量上升，总成本不降反升。**

【2010–2011 期末 B】结构化设计方法中，进行变换映射时，数据流边界的选择不是唯一的。

√ 正确。

解析：变换分析中传入流、传出流与主干变换的分界由设计者根据语义理解划分，不同切分得到不同初始模块结构。

【2010–2011 期中 B】类图和包图是对系统静态结构的描述。

√ 正确。

解析：类图、对象图、包图、组件图均属静态结构图；动态行为用顺序图、协作图、状态图、活动图。

【2010–2011 期中 B】**设计模型中的交互图是决定系统静态结构的基础和前提。**

√ 正确。

解析：§7.3——“**面向对象设计的最终结果是软件设计模型的静态结构，为了得到它，必须先得到软件设计模型的动态结构**”。交互图是静态结构（设计类图）的基础。

【2012–2013 期末 B】模块的耦合特性决定了软件的复杂程度。

√ 正确。

解析：耦合度上升直接抬高接口复杂度、修改传播范围和测试难度，决定软件整体复杂程度。

【2012–2013 期末 B】模块的内聚特性也可以应用于面向对象的软件设计。

√ 正确。

解析：§6.6.3——“无论是结构化，还是面向对象的程序，一个模块（或是一个类）内部的功能具有相同或相似的目的，可以认为该模块（类）具有较高的内聚性”。

【2012–2013 期末 B】**OOD 中，寻找对象职责的有效方法之一是绘制交互图。**

√ 正确。

解析：交互图把对象间消息（= 方法调用 = 职责承担）逐条画出，是发现/分配职责的有效手段。

【2012–2013 期末 B】面向对象的设计原则中要求先考虑使用继承复用，再考虑使用组合/聚合复用。

× 错误。

解析：§6.7.6 CARP 原文——“**实现复用时应首先使用组合/聚合，其次才考虑继承**”。题干顺序颠倒。

【2012–2013 期末 B】**某个软件对象的状态图可以表示该对象的子状态迁移。**

√ 正确。

解析：UML 状态图支持复合状态（Composite State）和子状态，可形成层次化状态机。

【2011–2012 期末 B】OOA 和 OOD 分别采用不同的概念和表示法。

× 错误。

解析：OOA 与 OOD 采用**一致的表示法（UML）**，二者无缝衔接。

【2011–2012 期末 B】**面向对象设计中最关键的活动是正确地给对象分配职责。**

√ 正确。

解析：§7.3 原文——“面向对象设计最关键的活动之一是**正确地给对象分配职责**”。

【2011–2012 期末 B】模块划分得越小，总的软件开发成本就越小。

× 错误。

解析：图 6–2 曲线表明存在最佳模块数 M 使总成本最小，并非“越小越好”。

【2011-2012 期末 B】 在进行变换映射时，数据流边界的选择是唯一的。

× 错误。

解析：变换分析的数据流边界选择不唯一，可由设计者根据语义切分。

【2011-2012 期末 B】 在协作图中，对象 A 接收了对象 B 一条消息，那么表明对象 B 具备了处理该条消息的职责。

× 错误。

解析：接收消息的对象（题干中的 A）才具有处理该消息的职责。题干将接收者和发送者颠倒。

【2013-2014 期末 A】 软件的概要设计的目的是为了确定软件的基本结构，方便后续的开发。

√ 正确。

解析：§6.3 概要设计要确定系统功能模型、数据模型及行为模型，明确软件的基本结构，为详细设计与编码提供框架。

【2018-2019 期末 B】 功能内聚比序列通信内聚的模块内聚程度更高。

√ 正确。

解析：§6.6.3 按内聚程度由低到高排序：巧合 < 逻辑 < 时间 < 过程 < 通信 < 顺序 < 功能。功能内聚是内聚程度最强的模块。

【2018-2019 期末 B】 在 OOD 方法中，模型的动态结构是进行模型静态结构设计的必要基础。

√ 正确。

解析：§7.3 原文——“面向对象设计的最终结果是软件设计模型的静态结构，为了得到它，必须先得到软件设计模型的动态结构”。先动态、后静态。

【2009-2010 期末 A】 面向对象的设计过程中，类图和包图就可以表示软件结构。

× 错误。

解析：设计模型需表达静态结构与动态行为两方面。类图、包图仅刻画静态结构，动态行为还需顺序图、协作图、状态图、活动图。

【2023-2024 期末 A】 软件的概要设计包括系统的界面设计和数据设计。

√ 正确。

解析：§6.3 概要设计 8 项活动中明确包含 6.3.4 数据设计 和 6.3.6 界面设计。

【2023-2024 期末 A】 模块的内聚描述的是类之间关系的紧密程度。

× 错误。

解析：内聚描述模块（或类）内部各元素结合的紧密程度；耦合才是描述模块之间（或类之间）的紧密程度。

二、单选题

【2007-2008 期末】 软件结构图中模块框之间的直线连接表示它们之间存在（ ）。A 调用关系 B 组成关系 C 链接关系 D 顺序执行关系

答案：A 调用关系。

解析：§6.8.3 SC 图中模块用矩形框表示，单向箭头（直线）从调用模块指向被调用模块，表示调用关系。

【2007-2008 期末】 为了提高模块的独立性，模块之间最好是（ ）。A 控制耦合 B 公共耦合 C 内容耦合 D 数据耦合

答案：D 数据耦合。

解析：数据耦合通过简单数据参数交换信息，是松散耦合，模块独立性强。控制/公共/内容耦合均为强耦合。

【2007-2008 期末】 在基于数据库的信息管理系统中，数据库概念模型的设计对应于系统开发的（ ）阶段。A 需求分析 B 概要设计 C 详细设计 D 程序设计

答案：B 概要设计。

解析：§6.3.4 数据设计是概要设计 8 项活动之一，确定数据库模式、子模式属于概要设计阶段。

【2010-2011 期中 A】 面向对象设计模型中不包括（ ）。A 操作契约 B 类的方法 C 用例实现的顺序图 D 对象的状态图

答案：A 操作契约。

解析：设计模型 = 动态结构（顺序图/协作图）+ 静态结构（设计类图，含类的方法/属性）+ 状态图/活动图。操作契约是 OOA 阶段用例模型的附加内容，作为设计输入而非设计模型本身。

【2010-2011 期末 B】 结构化软件设计是（ ）。A 面向数据结构的 B 面向数据库的 C 面向数据流的 D 面向对象设计的

答案：C 面向数据流的。

解析：结构化设计（SD）以 DFD 为输入，通过变换/事务分析映射为 SC 图，是面向数据流的设计方法。

【2010-2011 期末 B】 模块具有下面哪一种内聚性其独立性最弱？A 巧合内聚 C 信息内聚 D 通信内聚

答案：A 巧合内聚。

解析：§6.6.3 巧合内聚是内聚程度最低的模块。内聚程度越低，独立性越弱。

【2010-2011 期末 B】 软件详细设计的主要任务是确定每个模块的（ ）。A 算法和使用的数据结构 B 外部接口 C 功能 D 编程

答案：A 算法和使用的数据结构。

解析：§6.4 详细设计须完成——“确定软件各功能模块内的算法及各功能模块的内部数据结构”。外部接口、功能、总体结构属概要设计；编程属实现阶段。

【2010-2011 期末 B】 下面关于面向对象设计描述错误的是（ ）。A 需要考虑与具体实现有关的问题 B 与 OOA 采用一致的表示法 C GRASP 包含信息专家、创建者和控制器模式 D 用例实现方案设计在软件体系结构设计之前

答案：D（描述错误）。

解析：§7.2 先进行软件体系结构设计（确定分层结构），再做用例实现方案设计。D 把顺序说反了。

【2010-2011 期末 B】 模块的作用范围应在控制范围之内，下面哪一种做法是不当的？A 将判定所在模块合并到父模块使判定处于较高层次 B 将受影响模块下移到控制范围 C 将判定所在模块下移到最低层次 D 将判定上移到较高位置

答案：C（不当）。

解析：C “将判定下移到最低层次”使判定远离受影响模块，反而扩大作用范围超出控制范围，属反向操作。正确做法是 A/D（判定上移）或 B（受影响模块下移）。

【2011-2012 期末 B】 结构化软件设计与 OO 软件设计的关系？（ ）A 没有关系 B 本质一样但方法不同 C 结构化是 OO 一部分 D OO 是结构化一部分

答案：B 本质一样但方法不同。

解析：§6.3.1 两种方法在设计活动层面本质相同（都需完成 8 项活动），但采用的概念和表示法不同。

【2011-2012 期末 B】 面向对象的详细设计特指（ ）。A 对象的类设计 B 对象之间的调用关系设计 C 每个对象的属性定义 D 每个对象的方法定义

答案：D 每个对象的方法定义。

解析：§7.3 OO 详细设计核心是把每个对象的方法具体定义出来（方法名、参数、返回值、算法逻辑）。

【2011-2012 期末 B】 面向对象设计原则中的开闭原则是指 ()。A 允许改动原有方法定义 B 允许改动类的定义 C 只允许添加新功能 D 不允许添加也不允许改动

答案: C 只允许添加新功能。

解析: §6.7.2 OCP = 对扩展开放 (允许添加新功能)、对修改封闭 (不允许修改原有代码)。

【2011-2012 期末 B】 结构化的功能结构图中, 传入模块的结构有 ()。A 一个上级模块 B 一个下属模块 C 一个处理模块 D 包含前三个

答案: D。

解析: 传入模块承担"取得数据"职责, 包含一个上级模块 (被调用)、一个下属模块 (取得下一级输入) 以及自身的处理逻辑。

【2011-2012 期末 B】 当模块的作用范围不在其控制范围之内, 需要进行 ()。A 将判定所在模块下移 B 将判定语句上移 C 判定所在模块上移但判定不上移 D 不做调整

答案: B 将判定语句上移。

解析: 最有效的调整是把判定语句上移到作用范围的最近公共祖先层, 使作用范围纳入控制范围之内。

【2012-2013 期末 B】 下面选项中哪一个不是面向对象的软件设计? () A 数据结构设计 B 数据模型设计 C 数据流设计 D 迪米特法则的应用

答案: C 数据流设计。

解析: 数据流设计是结构化分析/设计的核心工具, 与 OO 无关。

【2012-2013 期末 B】 面向对象的概要设计不包括下面哪一个图? () A 序列图 B 类图 C 状态图 D 活动图

答案: D 活动图。

解析: OO 概要设计核心是用例实现 (交互图) + 设计类图 + 复杂对象状态图。活动图用于方法内部算法逻辑, 属详细设计。

【2012-2013 期末 B】 下面关于 OO 设计描述错误的是 ()。A 需要参考领域模型的概念类 B 与 OOA 采用一致的表示法 C GRASP 包含信息专家、创建者、控制器 D 动态结构是通过静态结构确定后得到的

答案: D (描述错误)。

解析: §7.3 原文——"为了得到静态结构, 必须先得到动态结构"。D 把顺序说反了。

【2013-2014 期末 A】 下面哪一个元素不属于结构化数据流程图? () A 加工 B 数据流 C 数据存储 D 数据项

答案: D 数据项。

解析: DFD 4 种基本元素 = 加工、数据流、数据存储、外部实体 (源/汇)。数据项属数据字典 DD 的元素。

【2013-2014 期末 A】 下面关于数据流图描述不正确的是 ()。A 每个加工只能有一个输入流和一个输出流 B 数据流图是封闭的 C 父图与子图之间输入输出一致 D 外部实体可以只有一个

答案: A (不正确)。

解析: 一个加工可以有多个输入流和多个输出流, A 错误。B/C/D 均为 DFD 的正确性质。

【2013-2014 期末 A】 下面哪一个是面向对象的软件设计原则? () A 数据结构设计 B 数据模型设计 C 数据流设计 D 迪米特法则的应用

答案: D 迪米特法则的应用。

解析: §6.7 七大原则包括 SRP/OCP/LSP/DIP/ISP/CARP/迪米特法则。A/B/C 均非设计原则。

【2013-2014 期末 A】 下面关于面向对象设计描述正确的是 ()。A 需要参考领域模型的概念类 B 与 OOA 采用不同的表示法 C GRASP 创建者模式用于确定哪个软件对象需要被动态创建 D 动态结构是通过静态结构确定后得到的

答案: A。

解析: A 正确。B 错 (采用一致表示法); C 错 (创建者解决"对象如何创建", 非动态创建); D 错 (先动态后静态)。

【2018-2019 期末 B】 在 MVC 风格的软件体系结构中, () 是模型的功能之一。A 封装内部数据与状态 B 提供用户操作的接口 C 将输入事件转换成服务请求 D 建立视图

答案: A 封装内部数据与状态。

解析: Model 封装业务数据与状态; View 提供用户操作接口/建立视图 (B、D 属 View); Controller 接收输入并转为服务请求 (C 属 Controller)。

【2018-2019 期末 B】 以下哪一项不属于变换型数据处理问题工作流程的主要工作步骤? () A 取得数据 B 计算数据 C 变化数据 D 给出数据

答案: B 计算数据。

解析: 变换型三步流程 = 取得数据 → 变换数据 → 给出数据。"计算数据"不是独立步骤, 而是变换的具体操作。

【2023-2024 期末 A】 面向对象的设计原则中哪一项描述的是模块的耦合性? () A 迪米特法则 B 开闭原则 C 单一职责 D 组合/聚合复用

答案: A 迪米特法则。

解析: 迪米特法则 (最少知识原则) 直接约束对象间的耦合度。OCP 关注扩展封闭; SRP 关注职责内聚; CARP 关注复用方式。

【2023-2024 期末 A】 以下哪种情况最有可能违反迪米特法则? () A 工具类提供多个静态方法供其他类调用 B 模块中的类只和同模块相关类通信 C 类通过层层调用最终访问无直接关联的底层类 D 类实现接口只和接口方法交互

答案: C。

解析: C 是典型"火车式"调用 a.b().c().d(), 违反迪米特法则"不与陌生人说话"。

三、多选题

【2023-2024 期末 A】 面向对象的概要设计需要用到以下哪几项 UML 的图? () A 用例图 B 交互图 C 类图 D 系统顺序图

答案: A B C D (最完整方案)。

解析: OOD 三项工作——体系结构设计、用例实现方案设计、用户界面设计。用例实现使用交互图 (B), 最终输出设计类图 (C); 输入参考为用例图 (A) 和系统顺序图 (D) (来自 OOA 阶段)。“需要用到”涵盖输入和产出。

四、填空题

【2023-2024 期末 A】 软件的概要设计主要进行两项设计活动, 分别是 () 和数据设计。

答案: (软件) 系统结构设计。

解析: §6.2~6.3 概要设计核心活动归纳为系统结构设计 (含模块层次、接口、体系结构) 与数据设计 (文件结构、数据库模式) 两大主线。

【2023-2024 期末 A】 面向对象设计中交互图的主要任务是确定 () 并为其分配功能。

答案: 软件对象。

解析: 交互图核心任务是确定系统事件进入系统后被激活的软件对象, 并为每个对象分配职责/方法 (功能)。分层设计的入口亦可填"控制器对象"。

五、简答题

【2008–2009 期末】 阐述面向对象设计中依赖倒置原则的含义。

标准答案：

1. 核心定义 (§6.7.4)：① 高层模块不应依赖低层模块，二者都应依赖"抽象"；② "抽象"不应依赖细节，细节应依赖"抽象"。
2. "倒置"的含义：① 依赖关系倒置——让低层依赖高层定义的抽象接口，高层通过抽象接口使用低层服务；② 接口所有权倒置——通常是客户端拥有抽象接口，而服务类从这些抽象接口派生。
3. 启发式规则：依赖"抽象"而非具体类——任何变量不应持有指向具体类的指针/引用；任何类不应从具体类派生；任何方法不应覆盖基类已实现的方法。
4. 意义：DIP 是框架设计的核心原则，能提高高层模块的可复用性、降低层次耦合，使系统易于扩展和维护。

【2011–2012 期末 B】 面向对象分析阶段与面向对象设计阶段有什么不同？相对于结构化需求分析与设计，OO 的分析与设计有什么优势？

标准答案：

- 分析阶段 (OOA)：针对问题域 (现实世界)，把需求转化为面向对象概念描述的系统分析模型，便于理解问题域和系统职责。
- 设计阶段 (OOD)：须考虑与具体实现有关的问题，目标是产生符合实现条件的设计模型。
- 相对结构化的优势：OOD 模型以 OOA 模型为基础，且 OOA 和 OOD 采用一致的表示法 (都是 UML)，使得从 OOA 到 OOD 不存在转换，两者紧密衔接，大大降低从 OOA 过渡到 OOD 的难度、工作量和出错率。

【2011–2012 期末 B】 什么是模块的信息隐藏？独立性强的模块应该是高内聚低耦合的，请分别列举两种高内聚类型和两种低耦合类型。

标准答案：

- 信息隐藏 (§6.6.2)：每个模块的实现细节对于其它模块是隐蔽的；模块中所包含的信息 (包括数据和过程) 不允许其它不需要这些信息的模块使用。
- 两种高内聚：功能内聚、顺序内聚 (也可答通信内聚/信息内聚)。
- 两种低耦合：非直接耦合、数据耦合。

【2012–2013 期末 B】 面向对象设计需要做哪些工作？这些工作对应的成果模型是什么？

标准答案：

三方面工作：① 软件体系结构设计 → 成果：UML 包图；② 用例实现方案设计 → 成果：交互图 (顺序图/协作图) + 设计类图 + 复杂对象的状态图；③ 用户界面设计。

详细设计：用活动图/程序流程图进行方法内部逻辑设计。

【2012–2013 期末 B】 什么是模块的耦合性？请列举至少 3 种降低耦合性的方法。

标准答案：

耦合性：模块之间相对独立性 (互相连接的紧密程度) 的度量。

降低耦合的方法 (每答对 1 个 1 分，3 个即可)：① 模块间多传数据信息，少传或避免传控制信息；② 降低模块接口的复杂性；③ 减少接口信息传送数量；④ 以系统调用 (call) 方式代替直接引用；⑤ 传送信息的结构尽量简单 (以数据耦合代替标记耦合)；⑥ 把模块的通信信息放在缓冲区中。

【2013–2014 期末 A】 请列举出 UML 中至少 4 种可用于软件设计阶段的图。

标准答案：类图、对象图、顺序图 (时序图)、协作图、状态图、活动图、包图、组件图、部署图 (任 4+)。

【2013–2014 期末 A】 请列举出面向对象设计过程中常用的五层软件分层体系结构，并说明模型层次化的好处。

标准答案：

五层：① 用户界面层 ② 控制器层 (处理层) ③ 业务逻辑层 (领域层) ④ 持久化层 ⑤ 系统层。

层次化好处：① 增加软件健壮性，易于扩展和维护；② 增加软件可移植性；③ 把消息流动限制在一个方向上，减少类间耦合。

【2018–2019 期末 B】 给出面向对象设计所必须的已知条件？以及设计结果是什么？

标准答案：

已知条件 (输入)：① 用例模型 (含用例图、用例描述、SSD、操作契约)；② 领域模型 (UML 类图表达的概念类及关联)；③ 软件体系结构框架 (如 MVC、SSH、五层分层)；④ OO 设计原则与设计模式 (7 大原则、GRASP)。

设计结果 (输出)：① 动态结构：用例实现的顺序图/协作图；② 静态结构：设计类图 (含分层、属性、操作、关系)；③ 必要时含状态迁移图、用户界面设计等。

【2021-2022 期末 A】 请说明软件设计的两个阶段是如何区分的？请说明用例模型中操作契约的后置条件所表示的对象与软件设计中的软件对象的区别与关系。

标准答案：

两阶段区分 (§6.2)：

维度	概要设计	详细设计
目标	总体框架（功能/数据/行为模型）	各功能模块的算法 + 局部数据结构
抽象层次	高层（模块划分、接口）	低层（接近源代码）
关注	What & Where	How

操作契约后置条件的对象 vs 软件对象：

- 后置条件所表示的对象 = 领域模型中的概念类对象（分析阶段产物，对问题域业务概念的抽象，不是软件实现类）。后置条件描述这些概念类对象的状态变化（创建/消除、关联建立/消除、属性值修改）。
- 软件对象 = OOD 阶段产物，是软件分层结构中可实例化的设计类，包括由概念类转换而来 + 为实现新增（控制器、DAO、工厂等）。
- 关系：① 概念类是软件类的输入与参考；② 一对一映射或多对一；③ 概念类提供"了解型职责"，软件对象则被分配"行为型职责"（方法），遵循 GRASP；④ 概念类仅描述业务状态变化，软件对象还含技术职责。

【2023-2024 期末 A】 软件的概要设计有哪几个主要活动？

标准答案（8 项）：① 制订设计规范 ② 软件系统结构设计 ③ 处理方式设计（性能设计） ④ 数据设计 ⑤ 可靠性设计（质量设计） ⑥ 界面设计 ⑦ 编写概要设计阶段文档 ⑧ 概要设计评审。

【2009-2010 期中 A】 请说明软件功能结构图中可能存在的模块控制范围以及模块作用范围之间的关系。如果出现"复杂"结构请给出可能的优化建议。

标准答案：

定义：控制范围 = 模块本身 + 所有下属模块；作用范围 = 受该模块判定语句影响的所有模块。

关系原则：作用范围必须在控制范围之内，且判定模块应靠近受影响模块。

违反时的优化：① 将判定语句上移到作用范围的最近公共祖先；② 将受影响模块下移到判定所在控制子树；③ 将判定所在模块合并到父模块使判定处于较高层次。错误做法：将判定下移到最低层次。

【2007-2008 期末 A】 简述面向对象开发方法中 OOA 和 OOD 要完成的工作。

标准答案：

OOA 工作：针对问题域进行抽象，输出用例模型（用例图+用例描述+SSD+操作契约）和领域模型（UML 类图表达的概念类及关系）。

OOD 工作：基于 OOA 模型并考虑实现，完成三方面——① 软件体系结构设计（输出包图）；② 用例实现方案设计（输出交互图+设计类图+状态图）；③ 用户界面设计。必要时进行详细设计（活动图/流程图）。

⑨ A4 双栏 Cheat Sheet (考前速查)

设计目标 / 过程

- 设计 = 把"做什么" (需求) 转为"怎么做" (物理模型)。
- 概要设计 8 活动: 规范、系统结构、处理方式、数据、可靠性、界面、文档、评审。
- 详细设计 = 算法 + 内部数据结构 + 流程图/PDL + 评审。
- 设计模型 = 静态结构 (功能+数据) + 动态结构 (过程+协调)。
- 设计三部分 (技术): 系统结构设计 + 数据设计 + 过程设计。

模块化 / 信息隐藏

- 模块 3 属性: 功能、逻辑、状态。
- 模块设计 5 标准: 可分解 / 可组装 / 可理解 / 连续性 / 保护性。
- 信息隐藏 (Parnas 1972): 实现细节对其它模块隐蔽; 改动影响局限于模块内。
- 模块数 M 使总成本最小 (图 6-2) —— 非越小越好。
- 结构图 (SC): 直线 = 调用; 空心圆尾箭头 = 数据; 实心圆尾 = 控制; 菱形 = 条件; 弧形 = 循环。

内聚 (低→高)

巧合 < 逻辑 < 时间 < 过程 < 通信 < 顺序 < 功能

- 巧合: 无独立功能、为省空间凑成 → 最低
- 逻辑: 传控制参数选功能
- 时间: 同时段执行 (初始化)
- 过程: 流程图次序
- 通信: 相同输入/输出数据
- 顺序 (序列): 上游输出=下游输入
- 功能: 单一功能不可分 → 最高、最佳

耦合 (低→高)

非直接 < 数据 < 标记 < 控制 < 外部 < 公共 < 内容

- 非直接: 无直接关系, 仅由上级调度 → 最低
- 数据: 简单数据参数 (最佳耦合)
- 标记 (特征): 传记录/数据结构子结构
- 控制: 开关/标志选功能
- 外部: 共享全局简单变量
- 公共: 共享公共数据环境
- 内容: 访问内部数据、非正常入口 → 最高、最差

降低耦合的方法

1. 多传数据, 少传控制;
2. 降低接口复杂度 (参数少、call 调用、避免指针);
3. 减少参数数量;
4. call 代替直接引用;
5. 用数据耦合代替标记耦合;
6. 通信信息放缓冲区。

OO 七大原则

- SRP: 单一职责 = 单一变化原因。
- OCP: 对扩展开放, 对修改封闭。
- LSP: 子类能替换父类。
- DIP: 高层/低层均依赖抽象 (框架核心)。
- ISP: 多专用接口 > 通用接口。
- CARP: 先组合/聚合, 后继承 (has-a vs is-a)。
- LoD: 最少知识, 不与陌生人说话。

"模块耦合性"对应 LoD; "扩展/修改"对应 OCP; "复用方式"对应 CARP; "类内聚"对应 SRP。

DFD → SC 图 (结构化设计)

变换型 (线性): 传入 → 变换中心 → 传出

1. 复查 DFD;
2. 确定类型;
3. 划边界 (边界不唯一);
4. 建 SC: 主模块 + 取得/变换/给出;
5. 优化 (高扇入、低扇出)。

事务型 (放射): 输入 → 事务中心 → N 输出

1. 找事务中心;
2. 定逻辑输入/输出;
3. 建 SC: 主模块 + 取得 + 调度 (菱形) + N 分支 + 给出。

三步流程: 取得数据 → 变换数据 → 给出数据。

OOD 核心

- OOD 三活动: 体系结构 + 用例实现 + UI。
- 先动态后静态: 动态结构 (交互图) → 静态结构 (设计类图)。
- OOD 详细设计 = 每个对象的方法 (活动图/流程图)。
- OOD 最关键活动 = 给对象分配职责 (GRASP)。
- OOA vs OOD: 均用 UML 表示法, 无缝衔接。
- OO 概要设计不包括活动图 (活动图属详细设计)。

五层体系结构

界面层 → 控制器层 → 业务逻辑层 → 持久化层 → 系统层

好处: 健壮可扩展可维护 + 可移植 + 减少类间耦合。

MVC 三要素

- Model: 封装内核数据与状态; 通知 View。
- View: 显示模型数据; 一 Model 可多 View。
- Controller: 接受用户输入, 转为服务请求。

优点: 分解、易扩展、易演化、可运行时改变。缺点: 局限于 UI 领域。

体系结构风格识别

- UNIX shell = 管道/过滤器
- 编译器 = 管道/过滤器
- 结构化程序 = 主/子程序风格
- OO 系统 = 对象风格
- OSI / OS = 分层风格
- IDE / 消息总线 = 基于事件
- Web 应用 = C/S
- Java VM = 解释器 (虚拟机)
- DB / 语音识别 = 仓库 / 黑板
- 用户界面 = MVC

体系结构定义

Garlan & Shaw (主流): 体系结构 = {构件, 连接件, 约束}

三要素: 构件层次结构 + 构件交互方式 + 数据结构。

4 大定义: Booch={组织+元素+子系统+风格}; Bass={构件+属性+关系}; Shaw 4 模型 (结构/框架/动态/过程); Garlan & Shaw。

作用范围 ≤ 控制范围

- 控制范围 = 本身 + 所有下属模块。
- 作用范围 = 受判定影响的所有模块。
- 违反时优化: 判定上移 / 受影响模块下移 / 合并到父模块。
- 错误: 判定下移到最低层。

扇入扇出

- 扇出 = 模块直接调用的下属数（多扇出 = 控制多下属）。
- 扇入 = 调用某模块的模块数（多扇入 = 公用模块）。
- 优化原则：高扇入、低扇出。
- 深度 = 层次数；宽度 = 同层最大模块数。

关键判断速答

- "最好耦合" → 数据耦合。
- "最差耦合" → 内容耦合。
- "最高内聚" → 功能内聚。
- "最低内聚" → 巧合内聚。
- "内聚描述" → 模块内部。
- "耦合描述" → 模块之间。
- "DFD 是哪个阶段" → 需求分析（做什么）。
- "结构化是面向" → 数据流。
- "OOD 最关键" → 给对象分配职责。
- "OCP 关键词" → 抽象。
- "先动态后静态" → OOD 模型。